DAMHWA / CODE REVIEW

담화 앱
코드 변경 비교·해설서

기존 코드와 수정 코드, 변경 이유, 실행 조건 및 검증 범위

비교 대상: 첨부「코드 개선.zip」안의 Damhwa와 Damhwa-fixed

작성일: 2026년 10월 21일 | 분석 방식: 실제 파일 비교 및 로컬 검사

이 문서는 업로드된 수정본을 분석한 결과입니다. 이 문서를 작성하면서 앱 소스를 추가로 수정하지 않았습니다.

먼저 알아둘 결론

수정본은 사용자 구분, 일정 검증, 저장 재시도, 채팅 서버 연동을 보완했습니다. 타입 검사와 API 자동 테스트를 통과했지만, 실제 휴대폰 음성대화와 알림까지 정상임을 확정한 결과는 아닙니다.

현재 첨부본의 프론트 환경설정은 미리보기 모드입니다. 화면에 보이는 예시 대화·일정·신고 성공 표시를 실제 서버 처리의 증거로 사용하면 안 됩니다.

코드 예시는 원본 파일에서 직접 추출했습니다. 파일 경로와 줄 번호는 이 첨부본 기준이며, 팀의 최신 저장소와 다를 수 있습니다. 비밀값이 들어갈 수 있는 .env 내용은 수록하지 않았습니다.

문서 읽는 방법과 변경 요약

각 항목은 기존 문제 → 변경 내용 → 실제 코드 → 해석과 확인할 점 순서입니다. 코드의 줄 번호는 원본·수정본 각각의 파일에서 가져왔으며 전체 파일을 복사한 것은 아닙니다.

01-02 사용자 식별·접근 검사

고정 ID 제거, 익명 토큰, 소유권 확인

03-04 일정 API·중복 방지

서버 경로 통일, 서버 ID, upsert

05-06 일정 해석·사용자 검토

날짜·시간 검증, 취소 제외, 수정·제외 UI

07-08 저장 실패·대화 수명

저장 성공 후 반영, 오류 안내, 상태 초기화

09-10 채팅·신고·차단

DB 저장, 조회 커서 분리, 대상 메시지 확인

11-12 DB·음성 연결

마이그레이션, 요청 제한·타임아웃

13-14 미리보기·개인정보

예시 데이터 분기, 위치 정밀도·로그 축소

15-17 검증·실행·남은 과제

테스트 범위, 실행 순서, 정확한 보고 문구

앞선 설명에서 바로잡는 부분

원본 useRoomChat.ts에도 3초 주기 조회가 있었습니다. 이번 변경은 폴링 자체의 최초 도입이 아니라, 백엔드 채팅 API·DB 추가와 커서·실패 처리 보완입니다. 또한 요약 API를 새로 구현한 것이 아니라 사용자 발언 일부와 규칙 기반 일정 후보를 보여주는 구조입니다.

01 고정 사용자 ID 제거

기존 문제

화면 진입 시 createConversation('1')을 호출하므로 여러 기기가 같은 사용자 ID로 대화를 생성할 수 있었습니다. 대화 생성 실패 시 start() 호출까지 건너뛰었습니다.

변경 내용과 이유

수정본은 화면 진입 시 start()를 호출하고, 저장 시점에 대화 레코드를 생성합니다. 사용자 식별과 대화 시작을 분리해 데모 화면이 DB 장애에 막히는 문제를 줄였습니다.

기존 코드 | frontend/src/app/voice.tsx

```
58  const initializeConversation = async () => {
59    try {
60      const conversation = await api.createConversation('1');
61
62      conversationIdRef.current = conversation.id;
63
64      console.log('대화 생성 완료:', conversation.id);
65
66      start();
67    } catch (error) {
68      console.error('대화 생성 실패:', error);
69    }
```

발췌 위치: L58-L69. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/app/voice.tsx

```
97  const saveTranscript = async () => {
98    if (USE_MOCK) return;
99    if (!conversationIdRef.current) {
100     const conversation = await api.createConversation();
101     conversationIdRef.current = conversation.id;
102   }
103   for (let i = savedTurns.current; i < finishedTranscript.current.length; i++) {
104     const turn = finishedTranscript.current[i];
105     await api.createConversationMessage(conversationIdRef.current!, turn.role === 'user' ? 'elder' : 'ai', turn.text,  turn-${i} );
106     savedTurns.current = i + 1;
107   }
108 };
109
```

발췌 위치: L97-L109. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

start()를 먼저 부른다고 실제 음성이 서버 없이 되는 것은 아닙니다. 실서버 음성 연결은 여전히 인증·백엔드·외부 API가 필요합니다. 데모와 실제 연결을 구분해야 합니다.

02 익명 토큰과 소유권 검사

기존 문제

기존 대화 생성 컨트롤러는 요청 본문의 userId를 그대로 사용했습니다. 요청자가 전달한 값만으로는 그 사용자인지 증명할 수 없습니다.

변경 내용과 이유

앱 설치별 사용자 등록과 세션 토큰 저장을 추가하고 Authorization 헤더를 보냅니다. 서버는 토큰으로 사용자를 찾고 req.userId를 설정합니다. 외부 입력 ID를 신뢰하지 않고 서버가 확인한 사용자 ID를 사용하려는 변경입니다.

기존 코드 | backend/src/controllers/conversation.controller.ts

```
  7      await conversationService.createConversation({
  8        userId: BigInt(req.body.userId),
  9        title: req.body.title,
 10      });
 11
```

발췌 위치: L7-L11. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | backend/src/controllers/conversation.controller.ts

```
  8      await conversationService.createConversation({
  9        userId: req.userId,
 10        title: req.body.title,
 11      });
 12
```

발췌 위치: L8-L12. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | backend/src/middleware/session.ts

```
 5  export async function requireSession(req: Request, res: Response, next: NextFunction) {
 6    const token = req.header('authorization')?.replace(/^Bearer /, '');
 7    if (!token || !/^[a-f0-9]{64}$/.test(token)) { res.status(401).json({ message: '앱 사용자 등록이 필요합니다.' }); return; }
 8    const user = await prisma.user.findUnique({ where: { sessionToken: token } });
 9    if (!user) { res.status(401).json({ message: '사용자 등록 정보를 확인해주세요.' }); return; }
 10    req.userId = user.id;
 11    next();
 12  }
```

발췌 위치: L5-L12. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

ownConversation은 대화 ID와 사용자 ID를 함께 검사하고, ownUser는 URL의 사용자와 토큰 사용자가 같은지 확인합니다. 익명 토큰은 로그인·계정 복구 체계가 아닙니다. AsyncStorage 보관과 토큰 만료·폐기·재등록 흐름은 운영 전 별도 검토가 필요합니다.

03 일정 API 경로와 서버 ID 통일

기존 문제

프론트는 /schedules/confirm, /schedules/:id를 호출했지만 원본 서버는 /users/:userId/schedules 계열을 제공했습니다. 경로와 응답 규격이 맞지 않았습니다.

변경 내용과 이유

프론트 요청을 사용자별 경로로 맞추고, 수정 서버에는 /:userId/schedules/confirm 처리를 추가했습니다. 저장된 배열과 서버 ID를 받아 화면에 반영합니다. API 주소와 데이터 형식을 맞춰 저장·조회·삭제가 같은 일정을 가리키도록 한 변경입니다.

기존 코드 | frontend/src/api/index.ts

```
179  export async function confirmSchedules(
180    items: ScheduleItem[],
181  ): Promise<void> {
182    if (USE_MOCK || items.length === 0) {
183      return;
184    }
185
186    await request<void>('/schedules/confirm', {
187      method: 'POST',
188      body: JSON.stringify({ items }),
189    });
190  }
191
```

발췌 위치: L179-L191. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/api/index.ts

```
172  export async function confirmSchedules(items: ScheduleItem[]): Promise<ScheduleItem[]> {
173    if (USE_MOCK || items.length === 0) return items;
174    const user = await ensureUser();
175    const saved = await request<ScheduleItem[]>(  /users/${user.id}/schedules/confirm  , { method: 'POST', body: JSON.stringify({
items
         }) });
176    return saved.map((item) => ({ ...item, serverBacked: true }));
177  }
178  export async function listSchedules(): Promise<ScheduleItem[] | null> {
179    if (USE_MOCK) return null;
180    const user = await ensureUser();
181    const saved = await request<ScheduleItem[]>(  /users/${user.id}/schedules  );
182    return saved.map((item) => ({ ...item, serverBacked: true }));
183  }
```

발췌 위치: L172-L183. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

serverBacked는 서버에서 받은 일정이라는 표시입니다. 서버는 내부 scheduleTime을 프론트의 date 필드로 변환합니다. 프론트와 백엔드를 함께 적용해야 하며, 프론트만 교체하면 경로·인증 불일치가 생깁니다.

04 메시지와 일정의 중복 저장 방지

기존 문제

통신이 끊겨 응답을 받지 못했을 때 서버는 이미 저장했을 수 있습니다. 기존 create 호출을 반복하면 같은 내용이 추가될 수 있었습니다.

변경 내용과 이유

대화 메시지는 conversationId + clientId로 upsert합니다. 일정은 userId + clientId로 저장·갱신합니다. DB에 복합 유일 인덱스도 추가했습니다. 같은 요청의 재시도를 새 데이터 생성과 구분하기 위한 처리입니다.

기존 코드 | backend/src/repositories/message.repository.ts

```
3  export async function createMessage(data: {
4    conversationId: bigint;
5    role: "elder" | "ai";
6    content: string;
7  }) {
8    return prisma.message.create({
9      data,
10   });
11 }
12
13 export async function findMessagesByConversationId(
```

발췌 위치: L3-L13. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | backend/src/repositories/message.repository.ts

```
3  export async function createMessage(data: {
4    conversationId: bigint;
5    role: "elder" | "ai";
6    content: string;
7    clientId: string;
8  }) {
9    return prisma.message.upsert({
10     where: { conversationId_clientId: { conversationId: data.conversationId, clientId: data.clientId } },
11     create: data, update: {},
12   });
13 }
14
```

발췌 위치: L3-L14. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | backend/src/routes/schedule.routes.ts

```
20   const rows = await prisma.$transaction(items.map((x) => prisma.schedule.upsert({
21     where: { userId_clientId: { userId: req.userId, clientId: x.id } },
22     create: { userId: req.userId, clientId: x.id, title: x.title.trim(), scheduleTime: new Date(x.date), category: x.category, place: x.place },
23     update: { title: x.title.trim(), scheduleTime: new Date(x.date), category: x.category, place: x.place ?? null },
24   })));
25   res.json(rows.map(shape));
```

발췌 위치: L20-L25. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

같은 글이라는 이유만으로 중복을 지우는 것은 아닙니다. 동일 clientId를 재사용한 요청만 같은 작업으로 취급합니다. 일정은 재시도 시 수정값을 갱신하지만, 대화 메시지는 update: {}이므로 기존 본문을 유지합니다. 최초 대화 생성 요청 자체의 중복 방지까지 보장하는 것은 아닙니다.

05 날짜·시간·취소 표현 검증

기존 문제

원본은 분 범위와 오전 12시 변환 검사가 부족했고, Date의 자동 보정 때문에 없는 날짜가 다른 날짜로 바뀔 수 있었습니다. 취소·과거 표현도 명시적으로 제외하지 않았습니다.

변경 내용과 이유

시간 범위, 실제 월·일 일치 여부를 확인하고 명백한 부정·취소·과거 문장을 제외합니다. 불명확한 후보에는 needsConfirmation을 표시합니다. 틀린 값을 조용히 저장하는 대신 후보를 제외하거나 사용자가 확인하게 하기 위한 변경입니다.

기존 코드 | frontend/src/lib/scheduleParser.ts

```
43  if (Number.isNaN(hour) || hour > 24) return null;
44  const meridiem = match[1];
45  let minute = 0;
46  if (match[3] === '반') minute = 30;
47  else if (match[4]) minute = parseInt(match[4], 10);
48
49  if (meridiem === '오후' || meridiem === '저녁') {
50    if (hour < 12) hour += 12;
51  } else if (meridiem === '낮') {
```

발췌 위치: L43-L51. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/lib/scheduleParser.ts

```
43  if (Number.isNaN(hour) || hour > 23) return null;
44  const meridiem = match[1];
45  let minute = 0;
46  if (match[3] === '반') minute = 30;
47  else if (match[4]) minute = parseInt(match[4], 10);
48
49  if (minute > 59) return null;
50  if (meridiem && (hour < 1 || hour > 12)) return null;
51  if ((meridiem === '오전' || meridiem === '아침') && hour === 12) hour = 0;
52
53  if (meridiem === '오후' || meridiem === '저녁') {
54    if (hour < 12) hour += 12;
```

발췌 위치: L43-L54. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/lib/scheduleParser.ts

```
119    // Conservative exclusion: cancelled/negative/past mentions must not create appointments.
120    if (/취소|안\s*(가|갈|갔|해|할)|않|못\s*(가|갈|해|할)|가지\s*마|갔|다녀|왔|다녀|왔|어제|지난/.test(c)) continue;
121
122    const t = extractTime(c);
```

발췌 위치: L119-L122. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

예: 오전 12시 → 00:00, 오후 3시 70분 → 제외, 2월 31일 → 제외. 정규식은 문맥 이해가 아니므로 “취소하지 않았어”처럼 부정이 겹친 문장이나 여러 약속이 섞인 문장은 추가 평가가 필요합니다. 이 규칙을 완전한 자연어 이해로 표현하면 안 됩니다.

06 일정 후보 수정·제외와 확인 버튼

기존 문제

원본 결과 화면은 추출된 summary.pending을 그대로 표시하고 확인했습니다. 사용자가 날짜·시간 오류를 바로잡거나 특정 후보만 빼는 흐름이 부족했습니다.

변경 내용과 이유

후보를 로컬 상태로 관리하고 제목·날짜·시간 편집, 후보 제외를 추가했습니다. 수정 중이거나 확인이 필요한 후보가 남아 있으면 저장 버튼을 막습니다. 규칙 추출의 한계를 사용자 검토 단계로 보완한 것입니다.

기존 코드 | frontend/src/app/voice.tsx

```
438   const items = summary.pending;
439   return (
```

발췌 위치: L438-L439. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/app/voice.tsx

```
476   const applyEdit = () => {
477     const match = /^(\d{4})-(\d{2})-(\d{2}) (\d{2}):(\d{2})$/.exec(dateText);
478     const d = match ? new Date(+match[1], +match[2] - 1, +match[3], +match[4], +match[5]) : null;
479     if (!title.trim() || title.length > 100 || !d || !Number.isFinite(d.getTime()) || localDate(d.toISOString()) !== dateText || d.getTime() <=
        Date.now()) {
480       showDialog('일정 확인', '제목과 미래 날짜를 YYYY-MM-DD HH:mm 형식으로 입력해주세요.'); return;
481     }
482     setItems((prev) => prev.map((x) => x.id === editing ? { ...x, title: title.trim(), date: d.toISOString(), needsConfirmation: false } : x));
483     setEditing(null);
484   };
485   return (
```

발췌 위치: L476-L485. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/app/voice.tsx

```
555           <PrimaryButton title={busy ? "저장 중…" : "확인"} disabled={busy || !!editing || items.some((x) => x.needsConfirmation)}
onPress
        ={() => onConfirm(items)} />
```

발췌 위치: L555-L555. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

날짜는 YYYY-MM-DD HH:mm 형식을 요구하며, 다시 형식화한 값이 입력과 같은지도 비교해 없는 날짜·시간 보정을 막습니다. 미래 일정인지 프론트에서 확인합니다. 서버 일정 API는 Date.parse 가능 여부를 검사하지만 같은 수준의 미래 날짜 정책을 강제하지는 않으므로 운영 정책 정합성은 추가 과제입니다.

07 저장 실패 안내와 재시도

기존 문제

기존 confirm은 로컬 기록을 먼저 반영하고 서버 저장 실패를 빈 catch로 무시한 뒤 화면을 닫았습니다. 종료 중 메시지 저장 실패도 이후 결과 처리에 영향을 줄 수 있었습니다.

변경 내용과 이유

서버 저장을 await한 후 반환된 일정으로 상태를 반영합니다. 실패하면 결과 화면에 오류를 남기고 재시도합니다. 대화 결과 생성과 서버 저장 실패도 분리했습니다. 실패를 성공으로 오해하지 않고 사용자가 복구할 수 있도록 순서를 바꿨습니다.

기존 코드 | frontend/src/app/voice.tsx

```
133  const confirm = () => {
134    if (!summary) return;
135    confirmConversation(summary.pending, summary.summary, [...session.getTranscript()]);
136    api.confirmSchedules(summary.pending).catch(() => {});
137    if (notificationsEnabled) summary.pending.forEach((it) => scheduleReminder(it).catch(() => {}));
138    router.back();
139  };
140
```

발췌 위치: L133-L140. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/app/voice.tsx

```
142      const saved = await api.confirmSchedules(items);
143      confirmConversation(saved, summary.summary, [...finishedTranscript.current]);
144      if (notificationsEnabled) {
145        const results = await Promise.allSettled(saved.map((it) => scheduleReminder(it)));
146        if (results.some((r) => r.status === 'rejected')) showDialog('알림 확인', '일정은 저장했지만 일부 알림을 등록하지 못했어요. 기기 알림 권한을 확인해주세요.');
147      }
148      router.back();
149    } catch {
150      setScreenError('저장하지 못했어요. 연결을 확인한 뒤 확인을 다시 눌러주세요.');
151    } finally {
152      confirming.current = false;
153      setSaving(false);
154    }
155  };
156
```

발췌 위치: L142-L156. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

대화 저장과 일정 저장 전체가 하나의 DB 트랜잭션인 것은 아닙니다. 일부 단계가 먼저 성공할 수 있어 clientId와 저장 진행 위치가 재시도를 돕습니다. 알림 실패도 일정 저장 실패와 분리해 안내합니다. 네트워크 장애 중 종료·확인·재시도는 휴대폰에서 별도 점검해야 합니다.

08 새 대화 초기화와 늦은 이벤트 차단

기존 문제

원본 start()에는 대화 transcript 초기화가 없었고 alive만으로 이벤트를 걸러냈습니다. 같은 화면에서 새 연결을 만들면 이전 연결 이벤트를 구별하기 어려웠습니다.

변경 내용과 이유

시작할 때 transcript와 표시 상태를 비우고, 이벤트를 발생시킨 client가 현재 client인지 확인합니다. 종료 시 연결 참조를 비우고 stop()을 호출합니다. 이전 대화 내용이나 늦게 도착한 답변이 새 세션에 섞이지 않도록 한 변경입니다.

기존 코드 | frontend/src/voice/useVoiceSession.ts

```
25  const start = useCallback(async () => {
26    if (client.current) return;
27    setError(null);
28    setUserText('');
29    setAssistantText('');
30
31    const events: VoiceEvents = {
32      onState: (s) => alive.current && setState(s),
33      onUserTranscript: (text) => {
34        transcript.current.push({ role: 'user', text });
35        if (alive.current) {
36          setUserText(text);
37          setAssistantText('');
```

발췌 위치: L25-L37. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/voice/useVoiceSession.ts

```
46  const start = useCallback(async () => {
47    if (client.current) return;
48    transcript.current = [];
49    setTurns([]);
50    setState('connecting');
51    setError(null);
52    setUserText('');
53    setAssistantText('');
54
55    let c: RealtimeClient;
56    const active = () => alive.current && client.current === c;
57    const events: VoiceEvents = {
58      onState: (s) => active() && setState(s),
59      onUserTranscript: (text) => {
60        if (!active()) return;
61        transcript.current.push({ role: 'user', text });
62        if (active()) {
63          setUserText(text);
```

발췌 위치: L46-L63. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

화면 retry에서도 경과 시간, conversationId, savedTurns, finishedTranscript를 초기화합니다. 이것은 새 대화 상태 정리이며 기존 서버 대화를 삭제하는 기능은 아닙니다. 마이크·스피커 해제와 앱 이탈·복귀는 실제 기기에서 검증해야 합니다.

09 채팅 서버 저장과 조회 커서 보완

기존 문제

원본에도 3초 폴링은 있었습니다. 다만 append()가 수신·전송 응답 모두에서 lastId를 갱신해, 폴링 사이에 들어온 다른 사람의 메시지를 건너뛸 가능성이 있었습니다. 원본 서버에는 채팅 라우트도 없었습니다.

변경 내용과 이유

채팅 DB/API를 추가하고 전송 응답과 조회 커서를 분리했습니다. 커서는 GET으로 받은 마지막 메시지에서만 전진합니다. 실패 안내와 동일 clientId 재시도도 추가했습니다. 서로 다른 사용자 간 메시지를 저장·조회하면서 중간 메시지를 누락하지 않도록 한 변경입니다.

기존 코드 | frontend/src/hooks/useRoomChat.ts

```
25  const append = useCallback((msgs: ChatMessage[]) => {
26    if (msgs.length === 0) return;
27    setMessages((prev) => {
28      const seen = new Set(prev.map((m) => m.id));
29      return [...prev, ...msgs.filter((m) => !seen.has(m.id))];
30    });
31    lastId.current = msgs[msgs.length - 1].id;
32    msgs.filter((m) => !m.isMine).forEach((m) => onIncomingRef.current?.(m));
33  }, []);
34
```

발췌 위치: L25-L34. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/hooks/useRoomChat.ts

```
39    const items = await api.listMessages(roomId, reset ? undefined : cursor.current);
40    if (!mounted.current || room.current !== current || epoch !== generation.current) return;
41    if (reset) { seen.current.clear(); setMessages([]); cursor.current = undefined; }
42    append(items);
43    if (items.length) cursor.current = items[items.length - 1].id;
44    setError(null);
45  }, [roomId, append]);
```

발췌 위치: L39-L45. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/hooks/useRoomChat.ts

```
94        const sent = await api.sendMessage(roomId, t, clientId);
95        if (mounted.current && room.current === current) append([sent]);
96        // Polling cursor only advances on GET, never on POST. This prevents
97        // skipping another person's message between two polling cycles.
98      }
```

발췌 위치: L94-L98. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

예: 마지막 조회 ID=10 → 상대가 11 전송 → 내가 12 전송. 전송 응답으로 커서를 12로 올리면 11을 놓칩니다. 수정본은 조회 커서 10을 유지해 다음 GET에서 11·12를 가져오고 화면 중복은 ID로 제거합니다. WebSocket 기반 즉시 수신으로 바꾼 것은 아닙니다.

10 신고·차단을 대상 메시지에 연결

기존 문제

원본 프론트에는 신고·차단 요청 함수가 있었지만 원본 서버에는 실제 처리 라우트가 없었습니다. 대상 선택과 저장까지 연결되지 않았습니다.

변경 내용과 이유

상대 메시지를 길게 눌러 선택하고 messageId를 전달합니다. 서버는 해당 방의 다른 사용자 메시지인지 확인한 뒤 신고·차단 관계를 저장합니다. 신고 대상을 명확히 하고 차단한 사용자의 메시지를 이후 조회에서 제외하려는 변경입니다.

기존 코드 | frontend/src/api/index.ts

```
273  export async function reportOrBlock(
274    roomId: string,
275    kind: 'report' | 'block',
276    messageId?: string,
277  ): Promise<void> {
278    if (USE_MOCK) {
279      return;
280    }
281
282    await request<void>(
283        /rooms/${roomId}/${kind}  ,
284      {
285        method: 'POST',
```

발췌 위치: L273-L285. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | backend/src/routes/chat.routes.ts

```
 50  for (const kind of ['report', 'block'] as const) {
 51    router.post(  /:roomId/${kind}  , async (req, res) => {
 52      const id = String(req.body.messageId ?? '');
 53      if (!/^\d+$/.test(id)) { res.status(400).json({ message: '상대방의 메시지를 길게 눌러 선택해주세요.' }); return; }
 54      const { rows } = await db.query('SELECT * FROM chat_messages WHERE id=$1 AND "roomId"=$2 AND "userId"<>$3', [id, req.pa
       rams.roomId, String(req.userId)]);
 55      if (!rows[0]) { res.status(404).json({ message: '대상 메시지를 찾을 수 없습니다.' }); return; }
 56      if (kind === 'report') await db.query('INSERT INTO chat_reports ("reporterId", "messageId") VALUES ($1,$2)', [String(req.userId),
       id]);
 57      else await db.query('INSERT INTO chat_blocks ("userId", "blockedId") VALUES ($1,$2) ON CONFLICT DO NOTHING', [String(req.u
       serId), rows[0].userId]);
 58      res.status(204).send();
 59    });
 60  }
 61  export default router;
```

발췌 위치: L50-L61. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

신고 저장은 운영자가 처리했다는 뜻이 아닙니다. 관리자 화면은 포함되지 않았습니다. 실서버 차단은 조회 필터이고, 미리보기는 가짜 접수 후 화면에서 숨기는 흐름입니다. 두 모드의 성공 표시를 같은 수준의 검증으로 취급하면 안 됩니다.

11 DB 마이그레이션과 유일 인덱스

기존 문제

기존 DB에는 세션 토큰과 일정·메시지 재시도 식별자, 채팅 저장 테이블이 없었습니다. 코드만 바꾸면 필요한 컬럼과 테이블이 없어 실행 중 오류가 납니다.

변경 내용과 이유

추가 SQL 마이그레이션에서 users.sessionToken, schedules/messages.clientId, 일정 분류·장소 및 채팅 관련 테이블·인덱스를 만듭니다. 응용 코드의 인증·중복 방지·채팅 기능이 DB 구조와 함께 동작하도록 맞춥니다.

수정 코드 | backend/prisma/migrations/20261020020000_demo_fixes/migration.sql

```
ALTER TABLE "users" ADD COLUMN "sessionToken" TEXT;
CREATE UNIQUE INDEX "users_sessionToken_key" ON "users"("sessionToken");
ALTER TABLE "schedules" ADD COLUMN "clientId" TEXT;
ALTER TABLE "schedules" ADD COLUMN "category" TEXT NOT NULL DEFAULT '일정';
ALTER TABLE "schedules" ADD COLUMN "place" TEXT;
CREATE UNIQUE INDEX "schedules_userId_clientId_key" ON "schedules"("userId", "clientId");
ALTER TABLE "messages" ADD COLUMN "clientId" TEXT;
CREATE UNIQUE INDEX "messages_conversationId_clientId_key" ON "messages"("conversationId", "clientId");
CREATE TABLE "chat_rooms" (
 "id" TEXT PRIMARY KEY, "title" TEXT NOT NULL, "meets" TEXT NOT NULL,
 "lat" DOUBLE PRECISION, "lng" DOUBLE PRECISION,
 "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE "chat_messages" (
 "id" BIGSERIAL PRIMARY KEY, "roomId" TEXT NOT NULL REFERENCES "chat_rooms"("id") ON DELETE CASCADE,
 "userId" BIGINT NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
 "text" TEXT NOT NULL, "clientId" TEXT NOT NULL,
 "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP
```

발췌 위치: L1-L18. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

기존 사용자 행의 sessionToken은 자동 발급되지 않고 nullable입니다. 기존 ID를 앱의 새 익명 세션에 자동 연결하는 계정 이전 기능도 아닙니다. SQL에는 기존 사용자·대화·일정 삭제 명령이 없지만 운영 DB에 배포하기 전 백업과 실제 PostgreSQL 마이그레이션 확인이 필요합니다.

12 음성 연결 오류·제한·타임아웃

기존 문제

원본은 요청 상태를 콘솔에 많이 남겼고 키 미설정·과도한 세션 생성에 대한 안내와 제한이 부족했습니다. 연결 실패가 사용자에게 기술적인 문구로 전달될 수 있었습니다.

변경 내용과 이유

음성 세션 API에 키 검사, 사용자별 10분 내 10회 제한, 외부 요청 20초 타임아웃을 추가했습니다. 프론트는 연결 오류를 쉬운 문구로 바꿉니다. 응답 없는 대기와 반복 요청을 줄이고 사용자가 다음 행동을 이해하도록 돕는 보완입니다.

기존 코드 | backend/src/routes/realtime.routes.ts

```
11  router.post("/session", async (req, res) => {
12    try {
13      const sdp = req.body;
14
15      if (!sdp) {
16        res.status(400).json({
17          message: "SDP is required",
18        });
19        return;
20      }
```

발췌 위치: L11-L20. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | backend/src/routes/realtime.routes.ts

```
11  const WINDOW_MS = 10 * 60 * 1000;
12  const MAX_SESSIONS = 10;
13  const recent = new Map<string, number[]>();
14
15  router.post("/session", async (req, res) => {
16    if (!process.env.OPENAI_API_KEY) {
17      res.status(503).json({ message: "음성 대화 서버 설정이 아직 안 됐어요." });
18      return;
19    }
20    if (!req.body || typeof req.body !== "string") {
21      res.status(400).json({ message: "SDP is required" });
22      return;
23    }
24    const key = String(req.userId);
25    const now = Date.now();
```

발췌 위치: L11-L25. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

제한은 프로세스 메모리 Map 기준이므로 서버 재시작 시 초기화되고 여러 서버 간 공유되지 않습니다. 익명 사용자를 새로 발급받을 수 있어 강력한 남용 방지 체계도 아닙니다. Realtime 연결 코드는 존재하지만 실제 외부 음성 통화 성공은 이번 자동 테스트 범위 밖입니다.

13 미리보기 모드와 실제 기능의 경계

기존 문제

원본은 API 주소가 없을 때 mock 모드로 실행했습니다. 수정본은 별도 PREVIEW_MODE를 추가해 일정·채팅 예시를 화면에 채우고 권한 요청 등을 우회합니다.

변경 내용과 이유

PREVIEW_MODE가 true이거나 API_URL이 비어 있으면 USE_MOCK가 true입니다. 현재 첨부 설정은 미리보기 true이고 API 주소가 비어 있습니다. 백엔드·마이크 없이 화면 흐름을 쉽게 시연하기 위한 편의 기능입니다.

기존 코드 | frontend/src/api/config.ts

```
 1  /**
 2   * 백엔드 주소. .env 에 EXPO_PUBLIC_API_URL=https://... 를 넣으면 실서버 모드,
 3   * 비어 있으면 앱 안에서 가짜 데이터로 동작하는 목(mock) 모드입니다.
 4   */
 5  export const API_URL = (process.env.EXPO_PUBLIC_API_URL ?? '').trim().replace(/\/$/, '');
 6  export const USE_MOCK = API_URL.length === 0;
 7
 8  /** 반경 2km 이웃 매칭 */
 9  export const NEIGHBOR_RADIUS_KM = 2;
```

발췌 위치: L1-L9. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/api/config.ts

```
 13  export const PREVIEW_MODE = (process.env.EXPO_PUBLIC_PREVIEW_MODE ?? '').trim().toLowerCase() === 'true';
 14
 15  /** 미리보기 모드이거나 서버 주소가 없으면 서버를 부르지 않습니다. */
 16  export const USE_MOCK = PREVIEW_MODE || API_URL.length === 0;
 17
 18  /** 반경 2km 이웃 매칭 */
 19  export const NEIGHBOR_RADIUS_KM = 2;
```

발췌 위치: L13-L19. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | frontend/src/api/index.ts

```
 162   if (PREVIEW_MODE && pending.length === 0) {
 163     // 미리보기: AI 요약·일정 추출 결과 예시를 보여줌 (실제 연결 시 서버 요약 API 로 교체)
 164     return { summary: PREVIEW_SUMMARY, pending: previewSchedules().map((x) => ({ ...x, id: makeId() })) };
 165   }
 166   if (PREVIEW_MODE) {
 167     // 미리보기: AI 요약처럼 보이는 간단한 문장 (실제 연결 시 서버 요약 결과로 교체)
 168     return { summary:  다마와 오늘 하루 이야기를 나누셨어요. ${pending.map((x) => x.title).join(', ')} 일정이 있다고 하셨어요.  , pending };
 169   }
 170   return { summary: userText.join(' · ').slice(0, 80) || '오늘 나눈 이야기', pending };
 171 }
```

발췌 위치: L162-L171. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

결과 화면의 자연스러운 요약 문장은 미리보기용 예시일 수 있습니다. 실제 모드는 사용자 발언을 이어 붙여 80자로 자르고 규칙으로 일정을 추출합니다. LLM 요약 API를 추가한 것이 아닙니다. 미리보기에서 신고 성공·알림 권한 허용처럼 보이는 것도 실제 서버 접수나 알림 도착의 증거가 아닙니다.

14 위치·로그와 오류 안내 보완

기존 문제

위치와 오류 객체를 그대로 다루면 필요 이상의 상세 정보가 서버나 로그에 남을 수 있습니다. 사용자 화면에서는 오류 원인과 다음 행동이 보이지 않는 경우도 있었습니다.

변경 내용과 이유

전송 위치를 소수 셋째 자리로 반올림하고, 오류 로그는 이름·코드·짧은 첫 줄 중심으로 축소했습니다. 마이크·인터넷·서버 오류 안내도 쉬운 문장으로 나눴습니다. 정보 수집 정밀도를 줄이고 문제 상황을 사용자가 이해할 수 있게 하려는 변경입니다.

수정 코드 | frontend/src/hooks/useNeighborhood.ts

```
  8  const round = (v: number) => Math.round(v * 1000) / 1000;
  9
 10  /** 반경 2km 이웃 매칭용 위치 권한/좌표 (실제 매칭은 서버가 좌표로 수행) */
```

발췌 위치: L8-L10. 줄바꿈은 지면에 맞춰 조정했습니다.

수정 코드 | backend/src/lib/logError.ts

```
  5  export function logError(label: string, error: unknown): void {
  6    const e = error as { name?: string; code?: string; status?: number; message?: string } | undefined;
  7    const bodyParseError = e?.name === 'SyntaxError' || (e as { type?: string } | undefined)?.type === 'entity.parse.failed';
  8    // JSON 파싱 오류 메시지에는 요청 본문(대화 내용) 일부가 들어가므로 버림
  9    const message = !bodyParseError && typeof e?.message === 'string' ? e.message.split('\n')[0].slice(0, 120) : '';
 10    console.error(  [${label}]  , e?.name ?? 'Error', e?.code ?? e?.status ?? '', message);
 11    if (process.env.NODE_ENV === 'development' && error instanceof Error) console.error(error.stack?.split('\n').slice(1, 4).join('\n'));
 12  }
```

발췌 위치: L5-L12. 줄바꿈은 지면에 맞춰 조정했습니다.

해석과 확인할 점

소수 셋째 자리 반올림은 좌표 정밀도를 줄이지만 위치를 익명화하거나 정확히 100m 오차를 보장하지는 않습니다. 짧은 오류 메시지에도 민감정보가 남을 수 있어 완전한 비식별 처리로 표현하면 안 됩니다. 반경 2km는 방 목록 필터이며 GPS 인증이나 방별 접근 통제는 아닙니다.

15 직접 검증한 결과와 한계

작성 시 프론트·백엔드 타입 검사, 프론트 린트, API 자동 테스트를 다시 실행했습니다. 웹 export는 앞선 2026-10-20 검사에서 성공했습니다. 웹 빌드 성공은 브라우저의 모든 버튼 동작과 휴대폰 기능 검증을 대신하지 않습니다.

- 백엔드 npm run typecheck: 통과
- 프론트 npm run typecheck: 통과
- 프론트 npm run lint: 통과
- 백엔드 npm test: tests 9 / pass 9 / fail 0
- 웹 expo export --platform web: 2026-10-20 성공

자동 테스트가 실제로 확인하는 내용

- 오전·오후 12시, 유효하지 않은 시각, 존재하지 않는 날짜
- 취소·부정·과거 표현 제외와 불명확한 시간 확인 표시
- 초기·추가 SQL을 독립 PGlite 메모리 DB에 적용
- 익명 사용자 인증과 타 사용자 일정·대화 접근 제한
- 대화 메시지와 일정 재시도 중복 방지, 일정 조회·삭제
- 두 사용자 채팅 요청·응답, 커서 조회, 신고 저장·차단 필터
- 위치 반경 목록 필터와 푸시 토큰 저장 요청

통과가 보장하지 않는 것

일반 PostgreSQL 운영 서버 배포, 휴대폰 화면·권한 창, 실제 OpenAI 음성 응답·스피커 출력, 앱 백그라운드 복귀, 기기 알림 도착, 대규모 동시 접속은 확인하지 않았습니다. 9개는 최상위 4개와 하위 5개를 합한 Node 테스트 개수이며, 전체 가능한 상황을 9개로 모두 검증했다는 뜻이 아닙니다.

테스트 DB는 운영 DB와 분리됩니다. 사용자를 두 명 만들어 HTTP API를 검증했지만 휴대폰 두 대의 UI를 직접 눌러 시험한 것은 아닙니다.

16 실행 순서와 시연 점검

A. 화면만 확인하기

Node.js 22.13 이상을 준비하고 Damhwa-fixed/frontend를 VS Code로 엽니다. 터미널에서 아래 명령을 실행한 뒤 표시된 웹 주소로 접속합니다. 기본 포트는 8081이며 이미 사용 중이면 터미널 안내를 따릅니다.

npm ci

npm run preview

현재 미리보기 설정에서는 API 키·DB 없이 예시 화면을 확인합니다. 프로그램을 실행했다고 실제 음성 연결까지 확인된 것은 아닙니다.

B. 실제 서버 기능 확인하기

1. PostgreSQL을 준비하고 backend/.env에 DATABASE_URL, OPENAI_API_KEY, PORT를 설정합니다. 실제 비밀 값은 프론트에 넣지 않습니다.

2. backend에서 npm ci → npm run db:generate → npm run db:migrate → npm start 순서로 실행합니다.

3. frontend/.env에서 EXPO_PUBLIC_PREVIEW_MODE=false로 바꾸고 EXPO_PUBLIC_API_URL에 휴대폰에서 접근 가능한 백엔드 주소를 지정합니다. localhost는 휴대폰 자신의 주소이므로 PC의 LAN IP를 사용합니다.

4. 프론트를 캐시를 비우고 다시 시작합니다. 실제 음성은 기존 WebRTC 네이티브 모듈을 포함하는 개발 빌드에서 확인합니다. Expo Go만으로 실제 음성 기능을 보장하지 않습니다.

휴대폰에서 꼭 확인할 흐름

마이크 허용 → “내일 오후 3시 치과” 발화 → 대화 종료 → 후보 수정·제외 → 저장 → 달력 조회·삭제. 이어서 인터넷 차단·복구 후 재시도, 대화 재시작, 두 기기 채팅, 신고·차단, 알림 권한 거절·허용을 확인합니다.

17 남은 과제와 보고할 때의 표현

기능 완성도

AI 자동 요약, 이웃 간 음성통화, 관리자 신고 처리, 계정 로그인·복구, 서버 푸시 발송 작업자는 이번 변경에 포함되지 않았습니다.

일정 추출

부정의 부정, 여러 날짜·시간이 섞인 문장, 연도·시간대 경계 등은 추가 테스트가 필요합니다. 정규식 검증은 모델 수준 의미 이해를 보장하지 않습니다.

익명 인증

토큰 보호·만료·폐기와 401 복구 흐름, 신규 사용자 등록 남용 제한 등을 운영 전 보완할 필요가 있습니다.

저장 원자성

메시지 저장·일정 저장·기기 알림은 하나의 원자적 작업이 아닙니다. 최초 대화 생성 응답 유실도 별도 복구 정책을 검토해야 합니다.

채팅 운영

전체 메시지 이력 페이지네이션, 대규모 폴링 부하, 신고 처리 절차와 차단 해제 UX는 후속 검토 대상입니다.

마이그레이션

기존 사용자 토큰 연결·데이터 이전과 실제 PostgreSQL 배포 검증은 팀 환경에서 확인해야 합니다.

검증 사실에 맞는 보고 문장

“사용자 인증, 일정 검증·수정, 저장 재시도, 채팅 연동을 보완한 수정본을 검토했습니다. 프론트·백엔드 타입 검사, 프론트 린트 및 자동 테스트 9개를 통과했고 웹 빌드를 확인했습니다. 실제 휴대폰 음성대화와 알림은 추가 검증이 필요합니다.”

“데모·실서버 모두 안정적으로 구동” 또는 “모든 기능 정상”이라고 단정하기에는 확인 범위가 부족합니다. 미리보기 화면의 성공과 실제 서버 처리를 구분해 보고하는 것이 정확합니다.

부록 A 변경 파일 목록

Damhwa를 기준으로 Damhwa-fixed에서 추가·수정·삭제된 소스·설정·마이그레이션 파일입니다. node_modules, generated, .env, 잠금 파일, 자동 생성 캐시와 문서·패치 파일은 제외했습니다. 목록은 동일 이름 파일의 바이트 비교 결과입니다. 새 기능이 아니라 설정·스타일만 달라진 파일도 포함합니다.

[수정] backend/.env.example
[수정] backend/package.json
[추가] backend/prisma/migrations/20261020020000_demo_fixes/migration.sql
[수정] backend/prisma/schema.prisma
[수정] backend/src/app.ts
[수정] backend/src/controllers/conversation.controller.ts
[수정] backend/src/controllers/message.controller.ts
[수정] backend/src/controllers/schedule.controller.ts
[수정] backend/src/controllers/user.controller.ts
[추가] backend/src/lib/logError.ts
[추가] backend/src/middleware/session.ts
[수정] backend/src/repositories/message.repository.ts
[추가] backend/src/routes/chat.routes.ts
[수정] backend/src/routes/conversation.routes.ts
[수정] backend/src/routes/message.routes.ts
[수정] backend/src/routes/realtime.routes.ts
[수정] backend/src/routes/schedule.routes.ts
[수정] backend/src/services/message.service.ts
[추가] backend/tests/regression.test.ts
[수정] frontend/.env.example
[수정] frontend/package.json
[수정] frontend/src/api/config.ts
[수정] frontend/src/api/index.ts
[수정] frontend/src/app/(tabs)/community.tsx
[수정] frontend/src/app/(tabs)/index.tsx
[수정] frontend/src/app/(tabs)/settings.tsx
[수정] frontend/src/app/_layout.tsx
[수정] frontend/src/app/room/[id].tsx
[수정] frontend/src/app/schedule.tsx
[수정] frontend/src/app/voice.tsx
[추가] frontend/src/components/DialogHost.tsx
[수정] frontend/src/hooks/useNeighborhood.ts
[수정] frontend/src/hooks/useRoomChat.ts
[추가] frontend/src/lib/dialog.ts
[수정] frontend/src/lib/format.ts
[수정] frontend/src/lib/notifications.ts
[수정] frontend/src/lib/scheduleParser.ts
[추가] frontend/src/preview/mockData.ts
[수정] frontend/src/store/useAppStore.ts
[수정] frontend/src/types.ts
[수정] frontend/src/voice/mockRealtimeClient.ts
[수정] frontend/src/voice/openaiRealtimeClient.ts
[수정] frontend/src/voice/useVoiceSession.ts

총 43개: 추가 8개, 수정 35개, 삭제 0개

부록 B 근거 파일과 추적 방법

근거는 첨부 압축파일의 실제 코드입니다. 이 문서에는 저장소 최신 상태 또는 외부 문서로 확인하지 않은 기능을 추가 구현된 것으로 기술하지 않았습니다.

frontend/src/api/index.ts

수정본 SHA-256 앞 20자리: ea52cc78fc22273f46e6

frontend/src/app/voice.tsx

수정본 SHA-256 앞 20자리: 9e72577100614a52e38a

frontend/src/lib/scheduleParser.ts

수정본 SHA-256 앞 20자리: f65966bb9f3e426aa1ee

frontend/src/hooks/useRoomChat.ts

수정본 SHA-256 앞 20자리: 3d4f170e2f02fa5b66b6

frontend/src/voice/useVoiceSession.ts

수정본 SHA-256 앞 20자리: 993895c6584e13915c2e

backend/src/middleware/session.ts

수정본 SHA-256 앞 20자리: a2437313afc4f0c56a15

backend/src/routes/schedule.routes.ts

수정본 SHA-256 앞 20자리: b8e0c94c35762fdc7a38

backend/src/routes/chat.routes.ts

수정본 SHA-256 앞 20자리: f4728a6688e3f73383ed

backend/tests/regression.test.ts

수정본 SHA-256 앞 20자리: 6c319c89fe8e46cbee0e

전체 변경 내용을 직접 확인하려면

VS Code에서 원본 파일을 우클릭해 “비교를 위해 선택”을 누르고 수정본의 같은 파일에서 “선택한 항목과 비교”를 선택합니다. 문서의 발췌 밖 문맥과 전체 구현을 함께 확인할 수 있습니다. 첨부된 CHANGES.patch는 이전 수정 단계의 패치일 수 있으므로 최종 폴더 파일 간 비교를 우선합니다.

문서의 범위

이 PDF는 핵심 변경을 이해하고 검토하기 위한 해설서입니다. 전체 소스 복사본이나 전체 diff 덤프는 아닙니다. 코드 인용은 표시된 파일·줄 범위에서 직접 추출했으며 파일 내용 변경 시 줄 번호와 해시는 달라집니다.